

AI-Agriculture Project Documentation

1. Project Overview

AI Agriculture is an intelligent agricultural management system that uses Artificial Intelligence and Machine Learning to help farmers make better agricultural decisions.

The system provides the following core features:

- Crop Recommendation: Recommends the best crop to plant based on soil metrics and weather.
- Plant Disease Detection: Identifies plant diseases from images.
- Disease Risk Prediction: Predicts the risk of diseases occurring based on environmental factors.
- Yield Prediction: Predicts the expected crop yield.

2. System Architecture

The project follows a modern microservices-style architecture, separated into three main components:

1. Frontend (React + Vite): The user interface of the application, providing an interactive dashboard for users.
2. Backend (Node.js + Express): Handles business logic, authentication, and database interactions using MongoDB.
3. AI Service (Python + FastAPI): A dedicated machine learning service that exposes prediction models via REST APIs.

This separation of concerns ensures that the heavy AI computations do not block the main backend, and allows each component to scale independently.

3. Technologies Used

Frontend:

- React.js, JavaScript, HTML, CSS, Vite

Backend:

- Node.js, Express.js, MongoDB, Mongoose

AI-Agriculture Project Documentation

AI Service:

- Python, FastAPI, Uvicorn
- Scikit-learn, TensorFlow / Keras, Joblib, NumPy, Pandas

4. Machine Learning Models

The AI Service relies on four trained machine learning models:

1. `crop_recommendation_model.pkl`: A Random Forest Classifier predicting 22 crop classes based on N, P, K, temperature, humidity, pH, and rainfall.
2. `disease_risk_model.pkl`: Predicts the likelihood of disease outbreaks.
3. `tomato_disease_model.keras`: A deep learning vision model for classifying tomato plant diseases from leaf images.
4. `yield_prediction_model.pkl`: Predicts expected yields based on soil and climatic conditions.

These models run in the FastAPI service and provide fast inference via REST endpoints.

5. Installation and Setup

To run the system locally, each component must be started independently:

1. Frontend: Run ``npm install`` and ``npm run dev`` in the `/frontend` directory.
2. Backend: Run ``npm install`` and ``npm run dev`` in the `/backend` directory. (Requires MongoDB running).
3. AI Service: Create a virtual environment, run ``pip install -r requirements.txt``, and start the service with ``uvicorn app:app --reload``.

Ensure that the pre-trained ML models are downloaded and placed in the ``ai-service/models/`` directory before starting the AI Service.